Evaluación extráulica 1 de **Diseño de Estructuras de Datos.**
Fecha de entrega: *jueves 12 de noviembre de 2026.*

Refugio Animal

Un refugio animal alberga perros y gatos. Cuando se trata de adoptar una mascota de dicho refugio, se opera estrictamente con una política *first in, first out (FIFO).* Dicho de otro modo, aquel animal que sea el que más tiempo tiene esperando, será el siguiente en ser adoptado. Quienes adoptan no pueden escoger a cuál mascota llevarse. Sin embargo, sí pueden elegir si se llevan un perro, un gato, o a cualquiera. En caso de escoger perro o gato, la misma política aplicaría: habría que llevarse al perro o al gato que más tiempo tenga esperando.

Implemente una estructura de datos que soporte el mantenimiento de dicho refugio animal.

El archivo de cabecera (.h) deberá verse así:

```
typedef struct strPet* Pet;
typedef struct strCatAndDogQueue* CatAndDogQueue;
typedef enum { FALSE, TRUE } Bool;
typedef enum { CAT, DOG } Type;

Bool pet_isDog(Pet p);
Bool pet_isCat(Pet p);
int pet_getId(Pet p);
Pet pet_create(int id, Type t);

CatAndDogQueue catAndDogQueue_create();
int catAndDogQueue_size(CatAndDogQueue q);
Bool catAndDogQueue_isEmpty(CatAndDogQueue q);
void catAndDogQueue_enqueue(CatAndDogQueue q, Pet p);
Pet catAndDogQueue_dequeue(CatAndDogQueue q);
Pet catAndDogQueue_dequeueCat(CatAndDogQueue q);
Pet catAndDogQueue_dequeueDog(CatAndDogQueue q);
```

Las operaciones *dequeue, dequeueCat* y *dequeueDog*, respectivamente, remueven y devuelven a la siguiente mascota (independientemente de su tipo), al siguiente gato o al siguiente perro.

La estructura y funciones que implementan a la mascota (y que ustedes deberán incluir en su archivo de implementación o .c) son así:

```
struct strPet {
int id;
Type type;
};

Bool pet_isDog(Pet p) {
return p->type == DOG;
}

Bool pet_isCat(Pet p) {
return p->type == CAT;
}

int pet_getId(Pet p) {
return p->id;
}

Pet pet_create(int id, Type t) {
Pet pet = (Pet) malloc(sizeof(struct strPet));
pet->id = id;
pet->type = t;
return pet;
}
```

Restricciones:

- Use una sola estructura de datos para almacenar a todas las mascotas (no es válido, por ejemplo, usar una cola para almacenar a los perros y otra para almacenar a los gatos).
- Desencolar a una mascota siempre debe ocurrir en tiempo constante, independientemente de si se desencola a una mascota de cualquier tipo, a un gato, o a un perro. ¿Qué quiere decir esto? Que no hay que buscar en la colección cada que se desencole.

Para revisar sus proyectos, usaré un programa como (aunque no necesariamente igual que) el siguiente:

```
int main() {

Pet myDogs[10];
Pet myCats[10];
```

```
for (int i = 0; i < 10; i ++) {
myDogs[i] = pet_create(i + 1, DOG);
}

for (int i = 0; i < 10; i ++) {
myCats[i] = pet_create(i + 1, CAT);
}

CatAndDogQueue animalShelter = catAndDogQueue_create();

Pet currentPet;

catAndDogQueue_enqueue(animalShelter, myDogs[0]);

for (int i = 0; i < 10; i ++) {
catAndDogQueue_enqueue(animalShelter, myCats[i]);
}

catAndDogQueue_enqueue(animalShelter, myDogs[1]);

for (int i = 0; i < 3; i ++) {
currentPet = catAndDogQueue_dequeueDog(animalShelter);
if (currentPet != NULL) {
printf("Dog #%03d has been adopted. Woof!\n", pet_getId(currentPet));
} else {
printf("All dogs have been adopted!\n");
}
}

for (int i = 0; i < 10; i ++) {
currentPet = catAndDogQueue_dequeueCat(animalShelter);
printf("Cat #%03d has been adopted. Meow!\n", pet_getId(currentPet));
}

return 0;
}
```

La salida esperada sería así:

Dog #001 has been adopted. Woof!

Dog #002 has been adopted. Woof!
All dogs have been adopted!
Cat #001 has been adopted. Meow!
Cat #002 has been adopted. Meow!
Cat #003 has been adopted. Meow!
Cat #004 has been adopted. Meow!
Cat #005 has been adopted. Meow!
Cat #006 has been adopted. Meow!
Cat #007 has been adopted. Meow!
Cat #008 has been adopted. Meow!
Cat #009 has been adopted. Meow!
Cat #010 has been adopted. Meow!